Create Coupon API - Cypress Test Documentation

Overview

This document describes the comprehensive Cypress test suite for the **Create Coupon API** endpoint (`POST /api/admin/coupons/create`).

Test Configuration

- **Base URL**: `http://localhost:5000`
- **Endpoint**: `/api/admin/coupons/create`
- **Method**: POST
- **Database**: Cleared before and after each test for isolation

Test Setup

Each test creates a test store first since coupons require a valid store reference:

```
// Creates test store with unique timestamp
const timestamp = Date.now() + Math.random().toString(36).substr(2, 9);
cy.request('POST', '/api/admin/stores/create', {
  storeName: `Test Store ${timestamp}`,
  slug: `test-store-${timestamp}`,
  description: 'Test store for coupon tests',
  category: 'Electronics'
});
```

Test Cases (15 Total)

1. Success Case - Valid Coupon Creation

- **Purpose**: Verify successful coupon creation with valid data
- **Input**: Complete coupon data with all required fields

- **Expected**: 201 status, coupon object with _id, title, code, discount, timestamps
- **Validation**: Response structure and field values match input

2. Missing Required Fields

- **Purpose**: Test validation for missing required fields
- **Input**: Incomplete data (only description field)
- **Expected**: 400 or 500 status with error message
- **Validation**: Error response structure

3. Duplicate Coupon Code Validation

- **Purpose**: Test unique constraint on coupon codes
- **Input**: Two coupons with identical codes
- **Expected**: First succeeds (201), second fails (400/409) or succeeds if constraint not active
- **Validation**: Appropriate error handling for duplicates

4. Invalid Data Types

- **Purpose**: Test type validation for fields
- **Input**: Wrong data types (number for title, boolean for code, array for discount)
- **Expected**: 400 or 500 status with error message
- **Validation**: Type validation enforcement

5. Minimum Required Fields Only

- **Purpose**: Test creation with bare minimum data
- **Input**: Only required fields (title, code, discount, store, category)
- **Expected**: 201 status, successful creation
- **Validation**: Coupon created with minimal data

6. Maximum Field Lengths

- **Purpose**: Test field length limits
- **Input**: Very long strings (200 chars for title, 500 for description)
- **Expected**: 201 (accepted) or 400 (rejected based on validation)
- **Validation**: Length validation handling

7. **Empty String Values**

- **Purpose**: Test handling of empty strings
- **Input**: All fields as empty strings
- **Expected**: 201, 400, or 500 based on validation rules
- **Validation**: Empty string handling

8. **Special Characters in Fields**

- **Purpose**: Test special character support
- **Input**: Title and description with special chars (@#$%^&*())
- **Expected**: 201 status, successful creation
- **Validation**: Special characters preserved in response

9. **Unicode Characters Support**

- **Purpose**: Test international character and emoji support
- **Input**: Chinese characters (测试) and emojis (🎯 🛍️)
- **Expected**: 201 status, unicode characters preserved
- **Validation**: Unicode content integrity

10. **Invalid Discount Formats**

- **Purpose**: Test discount field validation
- **Input**: Invalid discount format string
- **Expected**: 201 (accepted) or 400 (rejected) based on validation
- **Validation**: Discount format validation

11. **Null Values Handling**

- **Purpose**: Test null value processing
- **Input**: Some fields set to null
- **Expected**: 201 or 400 based on null handling rules
- **Validation**: Null value processing

12. **Performance Testing - Response Time**

- **Purpose**: Ensure acceptable API response time
- **Input**: Standard coupon data

- **Expected**: 201 status within 3 seconds
- **Validation**: Response time < 3000ms

13. Concurrent Coupon Creation

- **Purpose**: Test handling of multiple simultaneous requests
- **Input**: Three sequential coupon creation requests
- **Expected**: All succeed with 201 status
- **Validation**: Concurrent request handling

14. Large Payload Handling

- **Purpose**: Test large request body processing
- **Input**: Very large description (1000 chars) and extra data (2000 chars)
- **Expected**: 201 (accepted), 400 (validation error), or 413 (payload too large)
- **Validation**: Large payload handling

15. Content-Type Validation

- **Purpose**: Test HTTP header validation
- **Input**: Same data with different Content-Type headers
- **Expected**:
  - Wrong Content-Type: 400, 415, or 500
  - Correct Content-Type: 201 success
- **Validation**: Header validation enforcement

Data Structure

Input Schema

```
{
  title: String,        // Coupon title
  code: String,         // Unique coupon code
  discount: String,     // Discount value (e.g., "20%")
  description: String,  // Optional description
  store: String,        // Store ID reference
  category: String      // Coupon category
}
```

Expected Response Schema

```
{
  _id: String,          // MongoDB ObjectId
  title: String,        // Coupon title
  code: String,         // Coupon code
  discount: String,     // Discount value
  description: String,  // Description (if provided)
  store: String,        // Store reference
  category: String,     // Category
  createdAt: Date,      // Creation timestamp
  updatedAt: Date       // Last update timestamp
}
```

Error Handling

Tests validate various error scenarios:

- **400 Bad Request**: Invalid data, validation errors
- **409 Conflict**: Duplicate coupon codes
- **413 Payload Too Large**: Oversized request body
- **415 Unsupported Media Type**: Wrong Content-Type header
- **500 Internal Server Error**: Server-side errors

Test Isolation

- Database cleared before each test (`beforeEach`)
- Database cleared after each test (`afterEach`)
- Unique timestamps used to prevent data conflicts
- Each test creates its own test store

Performance Expectations

- Response time: < 3 seconds
- Concurrent request handling: Sequential processing
- Large payload: Graceful handling up to reasonable limits

Validation Patterns

Tests use flexible validation accepting multiple status codes to handle:

- Implemented vs unimplemented features
- Different validation rule configurations
- Server state variations

This comprehensive test suite ensures the Create Coupon API handles all edge cases, validates input properly, and maintains good performance characteristics.